ANALYSIS OF ALGORITHMS LABORATORY

Experiment no 2

1. AIM

To design, implement, and verify an iterative Binary Search algorithm using the Divide and Conquer strategy, and evaluate its operational performance metrics.

2. THEORY

Searching is a primary operation in computer systems designed to locate a particular data attribute within a structural layout. Binary Search stands out as an optimal solution for linear arrays due to its logarithmic growth curve.

2.1 The Divide and Conquer Framework

Binary Search leverages the Divide and Conquer technique to narrow down the problem set systematically:

- **Divide:** Compute the exact midpoint coordinate of the present boundary interval, splitting the dataset into balanced sub-arrays.
- **Conquer:** Compare the value residing at the computed midpoint index with the target criterion. If they line up, terminate immediately. Otherwise, discard the irrelevant sub-array partition completely.
- **Combine:** For pure retrieval problems, no explicitly executed merge pattern is necessary since target state maps out isolated outcomes directly.

2.2 Fundamental Prerequisite

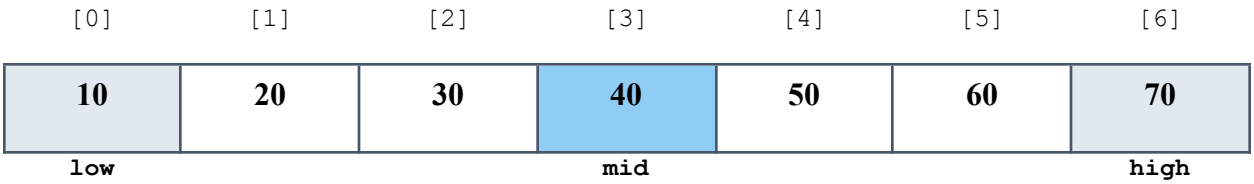
Binary Search strictly demands an explicitly ordered collection. Unsorted sets cannot support the directional assumptions that choose which side to drop.

2.3 Detailed Step-by-Step Graphical Simulation

Let us trace an iterative search for target key x = 20 inside a pre-sorted array with 7 elements.

Step 1: Initialization

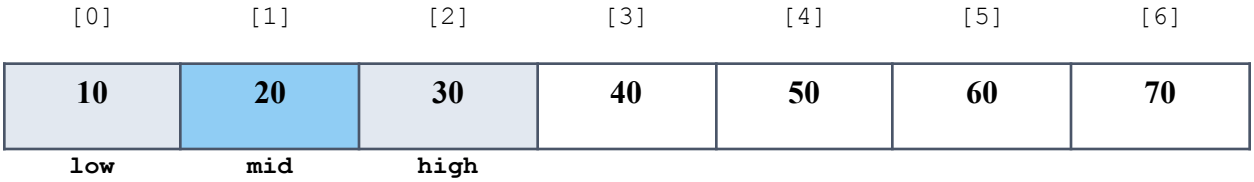
low = 0, high = 6
mid = (0 + 6) / 2 = 3
Value at mid (arr[3]) = 40



Action: Target 20 < 40. Therefore, discard the right partition and set high = mid - 1 = 2.

Step 2: Sub-array Processing

low = 0, high = 2
mid = (0 + 2) / 2 = 1
Value at mid (arr[1]) = 20



Action: Target 20 == 20. Target is identified at index 1. Execution completes successfully.

2.4 Algorithm Definition

```
Algorithm IterativeBinarySearch(arr, n, x)
// Input: Sorted array arr of size n, target key x
// Output: Index of x if found, else -1
Begin
    low ← 0
    high ← n - 1
    while low ≤ high do
        mid ← low + (high - low) / 2
        if arr[mid] == x then
            return mid
        else if arr[mid] < x then
            low ← mid + 1
        else
            high ← mid - 1
    endwhile
    return -1
End
```

2.5 Analysis Matrix

Metric Category	Complexity	Contextual Condition
Best Case Time Complexity	O(1)	Target found at first mid calculation match.
Average Case Time Complexity	$O(\log_2 n)$	Logarithmic pruning via split criteria.
Worst Case Time Complexity	$O(\log_2 n)$	Target at boundaries or not inside structural data.
Auxiliary Space Complexity	O(1)	Iterative processing maintains standalone tracking metrics without stack frames.

2.6 Practical Trade-offs & Application Environments

Advantages:
• Exceptionally efficient on massive arrays compared to O(n) linear lookup scans.
• Negligible memory footprint since standard tracking loop runs constant storage.

Disadvantages:
• Imposes an essential processing overhead because lists must remain pre-sorted.
• Array insertions and deletions introduce performance hits as sequential elements must shift.

Applications:
• Engine indexes inside modern transactional databases.
• Built-in internal system operations like 'bsearch' inside standard C language libraries.

3. C PROGRAM SOURCE CODE

```
#include <stdio.h>

// Function to perform iterative binary search
// Returns index of element if found, otherwise -1
```

```
int binarySearch(int arr[], int n, int x) {
    int low = 0, high = n - 1, mid;

    while (low <= high) {
        mid = (low + high) / 2;

        if (arr[mid] == x) {
            return mid;  // Element found at index mid
        } else if (arr[mid] < x) {
            low = mid + 1;  // Search in right half
        } else {
            high = mid - 1; // Search in left half
        }
    }

    return -1;  // Element not found
}

int main() {
    int n, i, x, result;

    printf("Enter number of elements (sorted array): ");
    scanf("%d", &n);

    int arr[n];
    printf("Enter %d sorted elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Enter element to search: ");
    scanf("%d", &x);

    result = binarySearch(arr, n, x);

    if (result == -1) {
        printf("Element %d not found in the array.\n", x);
    } else {
        printf("Element %d found at index %d.\n", x, result);
    }

    return 0;
}
```

4. SAMPLE INPUT & OUTPUT SCENARIOS

Scenario A: Element Found

Input:
Enter number of elements (sorted array): 7
Enter 7 sorted elements sequential order:
10 20 30 40 50 60 70
Enter element to search: 20

Expected Terminal Output:
Element 20 found at index 1.

Scenario B: Element Not Found

Input:
Enter number of elements (sorted array): 7
Enter 7 sorted elements sequential order:
10 20 30 40 50 60 70
Enter element to search: 99

Expected Terminal Output:
Element 99 not found in the array.

6. DRY RUN EXPLANATION

The execution matrix below outlines pointer updates for an array containing elements: [10, 20, 30, 40, 50, 60, 70] searching for x = 20:

Iteration	Low	High	Mid	Mid Value	Action Taken
1	0	6	3	40	20 < 40 → Shift high = mid - 1 = 2
2	0	2	1	20	20 == 20 → Match found. Return index 1

7. RESULT

The iterative Binary Search algorithm using the Divide and Conquer paradigm was designed, implemented, and verified successfully. The practical dry runs and complexity observations match theoretical models, confirming logarithmic runtime performance.